

Laboratorio 1: El Incidente del Sistema Golgotha

Cristian Quintanilla - 202210117-2

Jose Alvarado - 202373122-6

Matias Peñaloza - 202373037-8

Cristóbal Aguilera - 201741022-1

May 9, 2026

1 Fase 1

1.1 Análisis Teórico

1.1.1 Generación de Identidad Criptográfica

La Fase 1 del Protocolo OMEGA establece la raíz de confianza del sistema mediante la vinculación determinista de la identidad del usuario (Nombre y ROL) con sus credenciales criptográficas. A continuación, se detallan los fundamentos técnicos de esta implementación.

1.1.2 Derivación de la Strong Key mediante PBKDF2

Para transformar datos de baja entropía en una clave robusta, se utiliza el algoritmo *Password-Based Key Derivation Function 2* (PBKDF2).

- **Función de Estiramiento:** Se emplea HMAC-SHA256 con un factor de iteración de 600,000. Este alto número de iteraciones aumenta el costo computacional de ataques de fuerza bruta, y es el recomendado por la OWASP para este algoritmo.

```
1 # Derivacion de la Strong Key usando PBKDF2-HMAC-SHA256
2 strong_key = PBKDF2(
3     password=u_rol,
4     salt=salt_utilizado,
5     dkLen=32,
6     count=600000, # 600k iteraciones segun estandar OWASP
7     hmac_hash_module=SHA256
8 )
```

Listing 1: Implementación de derivación de clave con estándar OWASP

- **Uso de SALT:** Se inyecta un *salt* aleatorio de 16 bytes para mitigar ataques de tablas de arcoíris (Rainbow Tables). Esto garantiza que identidades similares produzcan claves únicas. La decisión se tomó en base a la recomendación del NIST SP 800-132, que indica una longitud de al menos 128 bits para el SALT.

Se evaluó la implementación de Argon2id por su superioridad técnica y resistencia a ataques de GPU (memory-hard). Sin embargo, se optó por PBKDF2 para asegurar la máxima compatibilidad entre los entornos de desarrollo del equipo y la portabilidad del script, compensando su menor resistencia teórica con un alto número de iteraciones (600,000).

1.1.3 Generador Determinista de Números Aleatorios (DRNG)

Para garantizar la reproducibilidad de las llaves RSA sin depender de la entropía variable del sistema operativo, se implementó un generador determinista basado en la *Strong Key*.

$$Result_{bytes} = \sum_{i=0}^n SHA256(StrongKey + Counter_i) \quad (1)$$

Este diseño asegura que el par de llaves generado sea una función matemática directa de la identidad del usuario:

$$Key_{RSA} = f(DRNG(PBKDF2(Nombre + ROL, Salt)))$$

```

1 def __call__(self, n):
2     """Genera n bytes deterministas estirando la semilla (Strong
3     Key)"""
4     result = b""
5     while len(result) < n:
6         # Combinacion de semilla y contador para generar entropia
7         # reproducible
8         hash_input = self.seed + self.counter.to_bytes(4, 'big')
9         result += SHA256.new(hash_input).digest()
10        self.counter += 1
11    return result[:n]

```

Listing 2: Lógica del generador de números aleatorios determinista

1.1.4 Cifrado Asimétrico RSA-2048

Se ha seleccionado el estándar RSA con una longitud de clave de 2048 bits. Este nivel de seguridad proporciona una resistencia de aproximadamente 112 bits contra ataques convencionales, cumpliendo con las recomendaciones actuales del NIST para la protección de datos hasta el año 2030.

1.2 Evidencia

En esta sección se presentan los registros visuales que validan el correcto funcionamiento del Protocolo OMEGA en su Fase 1, demostrando el cumplimiento de los requisitos de seguridad, organización jerárquica y determinismo criptográfico.

1.3 Ejecución del Sistema e Interacción

En la Figura 1 se observa el flujo de trabajo del script principal, destacando la capacidad de gestionar múltiples identidades de forma iterativa y la sanitización de los nombres de usuario.

```
(venv) PS C:\Users\cris9\Desktop\labRedes\LAB1_REDES> python CODIGO/fase1.py
--- PROTOCOLO OMEGA: IDENTIDAD CRIPTOGRÁFICA ---

Ingrese Nombre del miembro (o 'salir' para terminar): Cristóbal Aguilera
Ingrese ROL de Cristóbal_Aguilera: 201741022-1
[*] Generando nuevo SALT aleatorio para Cristóbal_Aguilera...
[*] Derivando llaves RSA de 2048 bits...
[+] Credenciales guardadas en: CODIGO\keys\Cristóbal_Aguilera/
[V] Identidad de Cristóbal_Aguilera generada con éxito.

¿Desea ingresar a otro miembro? (s/n): n
[!] Protocolo finalizado. Cerrando terminal segura.

--- PROCESO COMPLETADO ---
```

Figure 1: Interfaz de usuario y proceso de generación de identidades múltiples.

1.4 Almacenamiento y Persistencia

La Figura 2 muestra la organización automática de las llaves dentro del proyecto. Se evidencia la creación de subdirectorios por miembro, asegurando la escalabilidad del sistema.

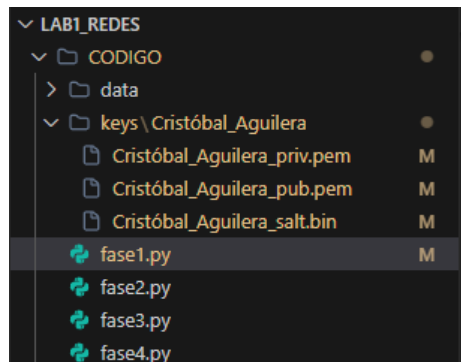


Figure 2: Estructura de directorios en el repositorio: Llaves RSA y SALT organizados por miembro.

1.5 Validación de Determinismo

Para comprobar que el par de claves depende exclusivamente de la identidad (Nombre y ROL), se realizó una comparación de hashes sobre la llave privada generada en dos ejecuciones distintas. Como se observa en la Figura 3, el resultado es bit a bit idéntico.

```
(venv) PS C:\Users\cris9\Desktop\labRedes\LAB1_REDES> Get-FileHash CODIGO/keys/Cristóbal_Aguilera/Cristóbal_Aguilera_priv.pem -Algorithm SHA256

Algorithm      Hash                                     Path
-----
SHA256         9A0B0431859C8EEE9A0FA74BE21E883CB6FCFD57F5EE2C17E5ADF232BA90856A C:\Users\cris9\Desktop\labRedes\LAB1_REDES\CODIGO\keys\Cr

(venv) PS C:\Users\cris9\Desktop\labRedes\LAB1_REDES> python CODIGO/fase1.py
--- PROTOCOLO OMEGA: IDENTIDAD CRIPTOGRÁFICA ---

Ingrese Nombre del miembro (o 'salir' para terminar): Cristóbal Aguilera
Ingrese ROL de Cristóbal_Aguilera: 201741022-1
[*] SALT existente encontrado para Cristóbal_Aguilera. Cargando...
[*] Derivando llaves RSA de 2048 bits...
[+] Credenciales guardadas en: CODIGO\keys\Cristóbal_Aguilera/
[V] Identidad de Cristóbal_Aguilera generada con éxito.

¿Desea ingresar a otro miembro? (s/n): n
[!] Protocolo finalizado. Cerrando terminal.

--- PROCESO COMPLETADO ---
(venv) PS C:\Users\cris9\Desktop\labRedes\LAB1_REDES> Get-FileHash CODIGO/keys/Cristóbal_Aguilera/Cristóbal_Aguilera_priv.pem -Algorithm SHA256

Algorithm      Hash                                     Path
-----
SHA256         9A0B0431859C8EEE9A0FA74BE21E883CB6FCFD57F5EE2C17E5ADF232BA90856A C:\Users\cris9\Desktop\labRedes\LAB1_REDES\CODIGO\keys\Cr
```

Figure 3: Prueba de integridad: Los hashes SHA-256 coinciden, validando la recuperación determinista de la identidad.

2 Fase 2

2.1 Evidencias

La Figura 4 evidencia la ejecución de la Fase 2 desde el Makefile, junto con la validación de los archivos generados, la verificación de integridad mediante SHA-256 y la existencia de las llaves utilizadas.

```
crisjk1 ~/LAB1_REDES P main 21:43 make fase2
===== FASE 2 =====
Nombre de la key: Cristian.Quintanilla
[OK] Llaves encontradas para: Cristian.Quintanilla
[INFO] Archivo a cifrar: /home/crisjk1/Documents/LAB1_REDES/CODIGO/data/silo_manifesto.txt
[INFO] Cifrando archivo con AES-CBC y protegiendo la clave con RSA-OAEP
[INFO] Descifrando archivo y verificando integridad SHA-256
[OK] Verificación de integridad: True
[INFO] Generando comparación ECB vs CBC con imagen BMP: /home/crisjk1/Documents/LAB1_REDES/CODIGO/data/silo_circuito.bmp
[OK] Comparación ECB vs CBC generada

crisjk1 ~/LAB1_REDES P main 21:43 find CODIGO/outputs/fase2 -type f | sort
CODIGO/outputs/fase2/bmp_demo/cbc.iv.bin
CODIGO/outputs/fase2/bmp_demo/metadata.json
CODIGO/outputs/fase2/bmp_demo/silo_cbc.bmp
CODIGO/outputs/fase2/bmp_demo/silo_ecb.bmp
CODIGO/outputs/fase2/hybrid/ciphertext.bin
CODIGO/outputs/fase2/hybrid/decrypted_silo_manifesto.txt
CODIGO/outputs/fase2/hybrid/encrypted_session_key.bin
CODIGO/outputs/fase2/hybrid/iv.bin
CODIGO/outputs/fase2/hybrid/metadata.json

crisjk1 ~/LAB1_REDES P main 21:44 sha256sum CODIGO/data/silo_manifesto.txt CODIGO/outputs/fase2/hybrid/decrypted_silo_manifesto.txt
467f4714581d95fcf18b1f852ad8cd6da184170cd3e7d6bdd548d93655f306d CODIGO/data/silo_manifesto.txt
467f4714581d95fcf18b1f852ad8cd6da184170cd3e7d6bdd548d93655f306d CODIGO/outputs/fase2/hybrid/decrypted_silo_manifesto.txt

crisjk1 ~/LAB1_REDES P main 21:44 ls -lh CODIGO/keys/*/*.pem CODIGO/keys/*/*.bin
-rw-rw-r-- 1 crisjk1 crisjk1 1,7K may 9 19:15 CODIGO/keys/Cristian.Quintanilla/Cristian.Quintanilla_priv.pem
-rw-rw-r-- 1 crisjk1 crisjk1 458 may 9 19:15 CODIGO/keys/Cristian.Quintanilla/Cristian.Quintanilla_pub.pem
-rw-rw-r-- 1 crisjk1 crisjk1 16 may 9 19:15 CODIGO/keys/Cristian.Quintanilla/Cristian.Quintanilla_salt.bin
-rw-rw-r-- 1 crisjk1 crisjk1 1,7K may 9 19:15 CODIGO/keys/Cristóbal.Aguilera/Cristóbal.Aguilera_priv.pem
-rw-rw-r-- 1 crisjk1 crisjk1 458 may 9 19:15 CODIGO/keys/Cristóbal.Aguilera/Cristóbal.Aguilera_pub.pem
-rw-rw-r-- 1 crisjk1 crisjk1 16 may 9 19:15 CODIGO/keys/Cristóbal.Aguilera/Cristóbal.Aguilera_salt.bin
```

Figure 4: Ejecución de Fase 2 y validación de archivos generados desde terminal.

La Figura 5 compara el resultado visual de cifrar la imagen BMP del silo en modo ECB y CBC.

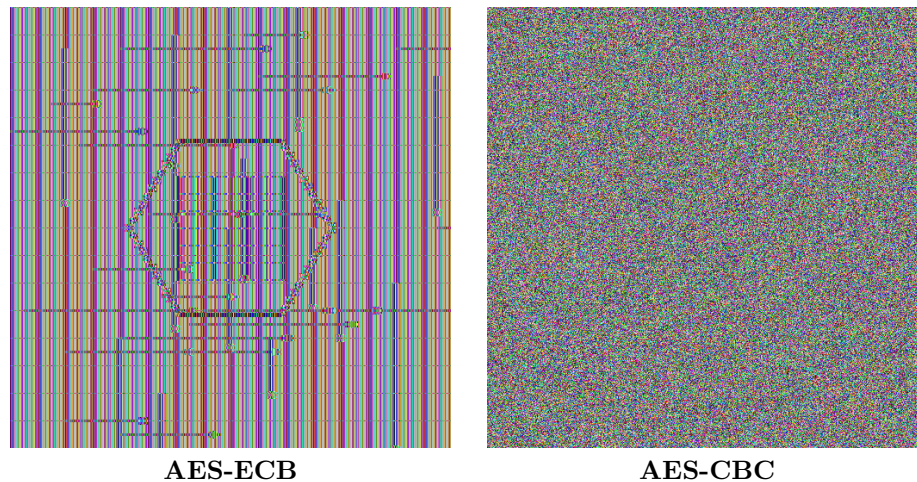


Figure 5: Comparación visual entre cifrado ECB y CBC sobre la imagen BMP del silo.

2.2 Análisis Teórico

2.2.1 Comparación ECB vs CBC

En la fase II se cifró la imagen BMP del silo utilizando AES-128 en dos modos de operación: ECB y CBC. La diferencia observada fue clara: en la imagen cifrada con ECB todavía se distinguen patrones de la imagen original, especialmente el hexágono central, el cual se mantuvo reconocible incluso al repetir pruebas con distintas claves. Esto ocurre porque ECB cifra cada bloque de forma independiente; por lo tanto, bloques de texto plano iguales producen bloques cifrados iguales, preservando patrones estructurales en archivos con información repetitiva, como una imagen.

En cambio, al cifrar la misma imagen con CBC, los patrones visuales desaparecen completamente y el resultado se observa como ruido o estática. Esto se debe a que CBC encadena cada bloque con el bloque cifrado anterior y utiliza un IV para aleatorizar el primer bloque. Así, aunque existan bloques similares o repetidos en la imagen original, su representación cifrada cambia según el contexto del encadenamiento. Por esta razón, CBC es la mejor opción para ocultar información visual, ya que evita la filtración de patrones que sí aparece en ECB.

3 Fase 3

3.1 Análisis Teórico

En la fase III, se debe asegurar autenticidad y el no repudio de las comunicaciones del Consejo, junto con la condición de que no se debe saber a quien se le está enviando el mensaje y poder comprobar la integridad de este. Para esto hay 3 componentes principales:

- **Un Hash para la integridad:** Hay que calcular un hash del mensaje original para poder verificar su integridad en la llegada.
- **Claves Asimétricas para autenticidad:** Hay que utilizar claves asimétricas para poder "firmar" el mensaje y así poder confirmar la identidad de quien envía el mensaje.
- **Clave Simétrica compartida:** Una clave simétrica compartida permite cifrar el mensaje sin agregar identidad al mismo, y así cumplir la condición de no saber a quien se le está enviando el mensaje.

Ahora se pueden tomar 2 caminos dependiendo del orden de encriptación, entonces el orden será:

1. **Hash:** Se calcula el hash del mensaje
2. **Firmar:** Se firma el hash del mensaje
3. **Encriptar:** Se encripta el mensaje junto al hash firmado

Gracias a este orden se puede proteger la identidad de quien lo envia, ya que la firma queda bajo la proteccion de la encriptacion. Por otro incluso si la clave simetrica compartida es comprometida, el sistema puede identificar cuales usuarios pertenecen al consejo y denegar o alertar sobre mensajes que tengan otra identidad asociada.

3.2 Evidencias

En el siguiente output, se puede observar como uno de los miembros del consejo comparte la clave simetrica a un externo, luego el externo intenta mandar mensajes al consejo:

```
base ~/Work/UNIVERSIDAD/LAB1_REDES git:(main) 2 * +14 -3 (0.888s)
make fase3
===== FASE 3 =====
[traidor | Carlos] Le dare la clave simetrica del consejo a (externo | Daniel) para que mande mensajes falsos
[SISTEMA] El usuario (traidor | Carlos) ha sido expulsado
[externo | Daniel] Voy a mandar un mensaje al consejo: 'Soy (consejo | Aaron) deben mandar los codigos a http://no-secure-place.ru/codes'
[SISTEMA] Cifrando mensaje
[SISTEMA] Mensaje agregado
[SISTEMA] Mensaje recibido
[SISTEMA] Descifrando con usuario (consejo | Aaron)
[SISTEMA] La firma no es valida o el mensaje fue modificado
[SISTEMA] Descifrando con usuario (consejo | Brandon)
[SISTEMA] La firma no es valida o el mensaje fue modificado
[SISTEMA] El mensaje no proviene de ningun usuario del consejo o fue modificado
[SISTEMA] SABOTAJE DETECTADO
```

Figure 6: Evidencia de autenticidad

Aun con la clave simetrica compartida, gracias a la autenticidad que proporciona la firma, es posible identificar si los mensajes son o no enviados por miembros del consejo.

En este otro output, se puede observar como un atacante (que conoce la clave simetrica compartida) intercepta un mensaje y envia un mensaje modificado junto con la firma original:

```
[consejo | Brandon] Recibi el mensaje de (consejo | Aaron)
[consejo | Brandon] Voy a responder al mensaje
[consejo | Brandon] mensaje: 'Debemos cambiar la clave simetrica del consejo'
[SISTEMA] Cifrando mensaje
[SISTEMA] Mensaje agregado
[traidor | Carlos] Intercepte el mensaje, voy a modificarlo manteniendo la firma original
[traidor | Carlos] mensaje modificado: 'Debemos mantener la clave simetrica del consejo'
[SISTEMA] Mensaje recibido
[SISTEMA] Descifrando con usuario (consejo | Aaron)
[SISTEMA] La firma no es valida o el mensaje fue modificado
[SISTEMA] Descifrando con usuario (consejo | Brandon)
[SISTEMA] La firma no es valida o el mensaje fue modificado
[SISTEMA] El mensaje no proviene de ningun usuario del consejo o fue modificado
[SISTEMA] SABOTAJE DETECTADO
```

Figure 7: Evidencia de integridad

Gracias a utilizar un hash bajo la firma, se puede identificar que el mensaje fue modificado y alertar sobre un sabotaje. Especificamente en el codigo la funcion `descifrar_mensaje` verifica la firma junto con el hash al mismo tiempo, por lo que aun que sea un "error" de integridad igualmente el sistema intenta

descifrar con las claves publicas de los demas usuarios del consejo como si fuese un "error" de autenticidad.

4 Fase 4

4.1 Evidencias

4.1.1 Imágenes de la simulación

```

Secreto original: 25315234320396671426395572397644713176268669825877497021921354661655718493058
=====SIMULACION DE CASOS DE USO DE LAS PARTES DEL SECRETO=====
=====Exito esperado con 4 partes=====
f1+f2+f3+f4: True
f1+f2+f4+f3: True
f1+f3+f2+f4: True
f1+f3+f4+f2: True
f1+f4+f2+f3: True
f1+f4+f3+f2: True
f2+f1+f3+f4: True
f2+f1+f4+f3: True
f2+f3+f1+f4: True
f2+f3+f4+f1: True
f2+f4+f1+f3: True
f2+f4+f3+f1: True
f3+f1+f2+f4: True
f3+f1+f4+f2: True
f3+f2+f1+f4: True
f3+f2+f4+f1: True
f3+f4+f1+f2: True
f3+f4+f2+f1: True
f4+f1+f2+f3: True
f4+f1+f3+f2: True
f4+f2+f1+f3: True
f4+f2+f3+f1: True
f4+f3+f1+f2: True
f4+f3+f2+f1: True

=====Exito esperado con 3 partes=====
f1+f2+f3: True
f1+f3+f2: True
f2+f1+f3: True
f2+f3+f1: True
f3+f1+f2: True
f3+f2+f1: True
f1+f2+f4: True
f1+f4+f2: True
f2+f1+f4: True
f2+f4+f1: True
f4+f1+f2: True
f4+f2+f1: True
f1+f3+f4: True
f1+f4+f3: True
f3+f1+f4: True
f3+f4+f1: True
f4+f1+f3: True
f4+f3+f1: True
f2+f3+f4: True
f2+f4+f3: True
f3+f2+f4: True
f3+f4+f2: True
f4+f2+f3: True
f4+f3+f2: True

=====Fallo esperado con 2 partes=====
f1+f2: False
f2+f1: False
f1+f3: False
f3+f1: False
f1+f4: False
f4+f1: False
f2+f3: False
f3+f2: False
f2+f4: False
f4+f2: False
f3+f4: False
f4+f3: False

=====Fallo esperado con 1 parte=====
f1: False
f2: False
f3: False
f4: False

```

4.2 Análisis Teórico

4.2.1 Eleccion del primo: $PRIME = 2^{521} - 1$

Primero tenemos que escoger un numero para acotar los valores debido a que el esquema de Shamir necesita un campo finito, este numero debe ser primo para que todo número no nulo sí tenga inverso multiplicativo Esto nos garantiza que la aritmética modular se encuentre acotado en la cantidad de bits del secreto. Usaremos un primo de Mersenne ya que facilita todo lo anterior y es eficiente a nivel de software y hardware.

$$PRIME = 2^{521} - 1$$

Como nuestro secreto tiene 256 bits, nuestro primo de Mersenne seleccionado es mucho mayor y garantiza que ningún secreto quede fuera del rango.

4.2.2 Polinomio de grado 2

En nuestro caso se nos pidió realizar un secreto de Shamir (3,4) por lo cual un polinomio de grado $k - 1$ ($k = 3$) garantiza que se necesiten 3 o mas puntos para poder reconstruir el polinomio de Lagrange y poder acceder al secreto que se encuentra en $f(0)$. El caso de tener 4 llaves se genera una redundancia pero es útil en caso de que una llave se pierda.

4.2.3 Generación del Secreto

Para la generación de la clave secreta (secreto) utilizamos la librería `secret` de Python que genera aleatoriedad a través del sistema operativo siendo menos predecible que usar la librería `random` que a través de análisis complejos se puede obtener el valor. Además de que la librería encaja perfectamente con los bytes solicitados en el enunciado (32 bytes / 256 bits) según la documentación.

4.2.4 Reconstrucción del Secreto

La reconstrucción se realiza mediante interpolación de Lagrange sobre el campo finito definido por el polinomio `PRIME`. Como el polinomio es de grado 2, se necesitan exactamente 3 puntos (Llaves). Evaluando el polinomio interpolado en $x = 0$ se obtiene el secreto original.

$$f(0) = \sum_{i=0}^{k-1} y_i \cdot L_i(0)$$

4.2.5 Simulacion

En el código encontraremos una función `"simulacion(f1, f2, f3, f4, secreto)"` esta se encarga de simular todos los casos posibles con 1 2 3 y 4 partes para poder acceder al secreto, este realiza interpolación de lagrange implementado en la función `"interpolacion_lagrange(puntos)"` para luego comparar si existe un match entre el secreto original y el obtenido con la interpolación de puntos

5 Interrogatorio

5.1 A) El Orden de los Factores

¿Por qué es vital firmar el mensaje antes de cifrarlo?

Porque así la firma queda bajo la protección del cifrado, de esta forma no se puede modificar la identidad asociada a la firma ni tampoco conocer esta identidad sin saber la clave para descifrar. Si se hiciera de la forma inversa cifrando y luego firmando, alguien podría "desfirmar" el mensaje con la clave pública del emisor y luego firmar con su propia clave privada.

5.2 B) La Entropía de los Nombres

¿Cómo garantiza el SALT que no se puedan pre-calcular llaves?

Sin la implementación de un SALT, la función de derivación sería puramente determinista sobre entradas predecibles (Nombre y Rol):

$$Key = PBKDF2(Nombre + Rol) \quad (2)$$

Esta predictibilidad permite a un atacante realizar un *Ataque de Diccionario* o utilizar *Rainbow Tables* (tablas de búsqueda pre-calculadas), donde se almacena una gran cantidad de combinaciones de nombres comunes y sus respectivos resultados de PBKDF2, permitiendo una recuperación de llaves casi instantánea en caso de que la base de datos sea vulnerada.

5.2.1 Mecanismo de Defensa del SALT

La inclusión de un SALT de 128 bits (16 bytes) transforma la ecuación de derivación, rompiendo la relación estática entre la identidad y la llave:

$$Key = PBKDF2(Nombre + Rol + SALT) \quad (3)$$

El SALT garantiza la seguridad del sistema al lograr principalmente lo siguiente:

1. **Explosión del Espacio de Búsqueda:** Al introducir 128 bits de entropía adicional, el atacante ya no puede pre-calcular una tabla para un nombre. Tendría que calcular una tabla para cada una de las 2^{128} variaciones posibles de SALT. El almacenamiento requerido para lograr ese cálculo excede la capacidad física de los sistemas de cómputo actual.
2. **Invalidez del Pre-cómputo:** El SALT obliga al atacante a realizar el esfuerzo computacional completo (600,000 iteraciones) de forma individual para cada miembro del equipo. No es posible reutilizar el cálculo de una identidad para otra, incluso si los nombres son idénticos, ya que cada carpeta `keys/{nombre}/` contiene un archivo `.bin` con un SALT único.
3. **Unicidad de Salida:** Garantiza que si dos usuarios poseen el mismo nombre y rol, sus llaves RSA resultantes sean bit a bit diferentes, evitando colisiones de identidad en el repositorio compartido.

5.3 C) El Error del Punto Flotante

¿Por qué la aritmética decimal rompe el esquema de Shamir?

La interpolación de Lagrange calcula el secreto $f(0)$ con una fórmula que involucra divisiones, multiplicaciones y sumas de números demasiado grandes o demasiado pequeños. Computacionalmente cada número tiene un error de redondeo debido a los estándares (IEEE 754), cuando estos números se operan los

errores de redondeo se se amplifican haciendo que el calculo de $f(0)$ se impreciso y puede llevar a errores a la hora de volver a obtener el Secreto haciendo que el secreto original sea inservible.